Assignment 3
PROG2053 Autumn 2026

Assignment Overview

This assignment contains basic static web development concepts. Submissions are due on **October 21, 2026.**
AI tools may be used for learning support only; please mention whether you have used them and for what purpose. All submitted work must be completed independently.

Instructions

If any part of this assignment is unclear, students are expected to make reasonable assumptions and clearly document them within their submission (e.g., as comments in the code or in an accompanying note). All stated assumptions will be considered during the marking process.

Please upload the assignment as '.txt' file.

Task 1. Callbacks

1. Create a master text function that applies formatting rules based on different callback styles.
   The Rules:
   - Main function: `formatText(str, callback)`
   - Callback 1: `toShouting(str)` (converts to uppercase with an exclamation mark).
   - Callback 2: `toSecret(str)` (replaces vowels with asterisks `*`).

Expected Outcome

```
HELLO WORLD!
h*ll* w*rld
```

2. The main function generates a random number between 1 and 10. The callback functions then perform a specific math action on that hidden number.
   - Double that number
   - Checking Even or Odd

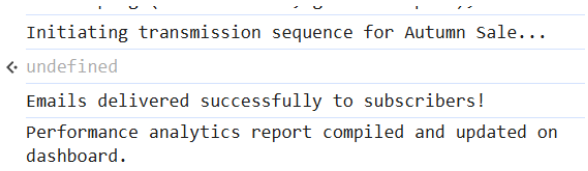
Expected Outcome

```
The random number is: 8
Result: 16
The random number is: 8
Result: Even
```

Task 2. setTimeout

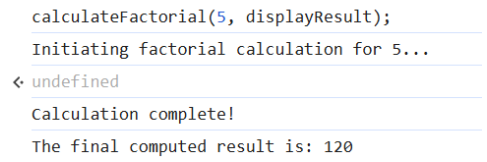
3. Create a system that simulates sending an automated email marketing campaign by writing a main function called sendCampaign(campaignName, callback) that logs "Initiating transmission sequence for [campaignName]...", uses setTimeout to simulate a 3-second network delivery delay before logging "Emails delivered successfully to subscribers!" and triggering the callback, defines a callback function called generateReport() that logs "Performance analytics report compiled and updated on dashboard.", and finally executes the code by passing "Autumn Sale" and generateReport as parameters.

Expected Outcome



4. Create a system that simulates a complex mathematical calculation by writing a main function called calculateFactorial(number, callback) that logs "Initiating factorial calculation for [number]...", uses setTimeout to simulate a 1.5-second processing delay before calculating the factorial value, logging "Calculation complete!" and triggering the callback with the computed result as an argument, defines a callback function called displayResult(result) that logs "The final computed result is: [result]", and finally executes the code by passing 5 and displayResult as parameters.

Expected Outcome



Task 3. Promise

5. Create a system that simulates rolling a six-sided die using a Promise by initializing a new Promise object assigned to a constant variable named diceRoll, utilizing Math.floor() and Math.random() inside its execution block to generate a random whole number between 1 and 6, resolving the promise with the message "High roll! You win." if the number is greater than 3, rejecting it with "Low roll! You lose." if the number is 3 or lower, and properly chaining .then(), .catch(), and .finally() handlers to log the respective outcome messages followed by a permanent "Game completed." execution message when settled.

Expected Outcome

```
Low roll! You lose.
Game completed.
```

6. Check whether a shopping cart coupon code is valid and calculate the discount if it meets the conditions. Consider the following steps:

   Step 1: Initialize a new Promise object and assign it to a constant variable named `validateCoupon`.
   Step 2: Inside the execution block of the promise, check a static `shoppingCart` data object containing a coupon string, an expiration status, and a base checkout price.
   Step 3: Set up the validation conditions:
   - If the coupon code does not match `"SAVE15"`, trigger the failure state by calling `reject()` with an invalid code error message.
   - If the coupon code is correct but the expiration status is `true`, trigger the failure state by calling `reject()` with an expired code error message.
   - If the coupon code matches `"SAVE15"` and is not expired, calculate a 15% savings deduction and trigger the fulfillment state by calling `resolve()` with a success message showing the final price.

   Step 4: Properly chain the `.then()`, `.catch()`, and `.finally()` block handlers to process the respective outcomes, log the resulting strings to the console, and log a permanent `"Session sync complete."` execution message when settled.

Expected Outcome

```
Success! Saved $30. Your adjusted checkout total is $170.
Session sync complete.
```

Task 4. Async/await

7. Create an Inventory Check System by considering the following steps:

   Step 1: Write an async function called `checkStockLevel()` that simply returns a numerical stock count (for example, 45).
   Step 2: Write an async function called `isWarehouseOpen()` that simply returns a boolean value (`true` or `false`).
   Step 3: Write a main async function called `processItem(itemName)` that accepts a string parameter and handles the following evaluation rules:
   1. Await the boolean value from `isWarehouseOpen()`. If it is `false`, print an operation aborted message and stop.
   2. If it is `true`, print a warehouse open message, then await the numerical value from `checkStockLevel()`.
   3. Add a buffer of `10` items to that stock count and log a personalized tracking message containing the `itemName` and total calculation.

   Step 4: Execute the system by calling your main function and passing a string argument (for example, `"Laptop"`).

Expected Outcome

```
System Log: Warehouse is open. Processing item...
Tracking: Laptop has a total buffered inventory count of 55 units.
```

8. Create a Subscription Upgrade System by considering the following steps:

   Step 1: Write an async function named `getLoyaltyDiscount(yearsActive)` that accepts a number representing how many years a user has been active.
   Step 2: Inside this first function, perform a boolean check (`yearsActive > 2`). If `true`, return a numerical discount of `15`. If `false`, return a numerical discount of `0`.
   Step 3: Write a second async function named `applyUpgrade(accountName, yearsActive)` that accepts a string for the user's name and a number for their active years.
   Step 4: Inside the second function, use exactly one `await` expression to call `getLoyaltyDiscount(yearsActive)` and store the result.
   Step 5: Subtract the returned discount value from a base charge of `50`, then log the final balance to the console alongside the user's account name.

Expected Outcome

```
applyUpgrade("Premium-User-Alpha", 4);
System Log: Starting upgrade process for Premium-User-Alpha...
Invoice: Account 'Premium-User-Alpha' processed. Final balance due: $35.
```

Task 5. GET & POST

9. Create a system that extracts travel booking parameters by writing a function called **getBookingInfo()** that initializes a full URL variable named **urlString** set to **"https://travel.com?price=1000&day=Wednesday"**, splits the string at the question mark and selects index **[1]** to isolate the query segment, splits it again at the ampersand symbol into separate property pairs, loops through the pair entries using a traditional index-based loop to break each part at the equals sign, and logs the resulting keys and values cleanly to the console.

Expected Outcome

```
Key is: price
Value is: 1000
Key is: day
Value is: Wednesday
```

10. Create an HTML form structure using following steps:
    Step 1: that handles a movie filter query by setting the target action to /filter.php and specifying the GET transmission method.

Step 2: Inside the form, add a text input field with the name attribute set to genre and a default value of action.
Step 3: Add a numeric input field with the name attribute set to year and a default value of 2025.
Step 4: Provide a submit button labeled "Filter Movies".
Step 5: Write down what the exact **resulting URL address** will look like in the browser bar after the user clicks the submit button with the default values.

11. Write a JavaScript function called getFlightInfo() that initializes a full URL variable named urlString set to "https://fly.com?name=norwegian&passenger=60", isolates the query data string by splitting at the question mark, instantiates a URLSearchParams utility object, utilizes a for...of loop over its entries to unpack the parameters, and outputs each structural key and value sequence to your logs.

Expected Outcome

```
Key is: name
Value is: norwegian
Key is: passenger
Value is: 60
```

12. Write a raw HTTP text block that represents the network packet generated by a web browser when a form containing checkout data is submitted to a server domain named **shop.com** at the endpoint **/checkout.php** using the POST method and a payload body containing the keys product_id=883 and qty=2 linked by an ampersand.

Task 6. Backend Development

13. The student registration module is being built using Node.js. Write a script utilizing the modern asynchronous **`fs/promises`** module that completes the following steps:

A. Create and Write Database: Create a brand-new file called `students.txt` and populate it with the names `Alice`, `Bob`, and `Charlie` on individual lines. Upon successful creation, output the message `"File created successfully."` to the console.
B. Read and Verify Roster: Immediately read the newly created `students.txt` file from disk and display its raw contents directly within the console terminal to verify accuracy.
C. Append New Record: Modify the file by appending the student name `David` to the end of `students.txt` without deleting or overwriting any of the existing three records. Ensure David begins cleanly on a fresh line.
D. Verify Database Existence: Programmatically verify whether the `students.txt` database file exists on the hard drive. If it is detected, print `"The file exists."` to the console; otherwise, print `"The file does not exist."`.

E. Archive and Rename File: Simulate an database archiving step by looking for a file named `oldstudents.txt` and renaming it to `students.txt`. After successfully changing the name, display the confirmation message `"File renamed successfully."`.

Expected Outcome

